

Curso de Automatización con Java, Selenium, JUnit y Gherkin

Clase 9 – Patrones de diseño en automatización

COLEGIO
UNIVERSITARIO



CCC
Centro de Capacitación
Continua

Temario

- ♦ Page Object Model (POM) avanzado
- ♦ Reutilización de código
- ♦ Clases base (BaseTest / BasePage)
- ♦ Centralización de esperas
- ♦ Utils y helpers

 **Práctica: refactorizar un flujo existente**

¿Por qué usar patrones de diseño en automatización?

Problemas comunes sin patrones:

- Código duplicado
- Tests largos y difíciles de leer
- Cambios en la UI rompen muchos tests
- Dificil mantenimiento
- Baja escalabilidad

👉 Los patrones de diseño ayudan a:

- Reutilizar código
- Separar responsabilidades
- Mejorar legibilidad
- Facilitar mantenimiento
- Trabajar en equipo

Repaso del Page Object Model (POM)

Concepto clave

- Cada página = una clase
- Los tests **NO** deben tener locators
- Los tests describen el **qué**
- Las páginas implementan el **cómo**

Mal:

```
driver.findElement(By.id("user-name")).sendKeys("user");
```

Bien:

```
loginPage.login("user", "pass");
```

Limitaciones del POM básico (Clase 8)

Aunque el POM básico funciona, tiene problemas:

- Cada Page repite `driver.findElement()`
- Cada Page maneja esperas por su cuenta
- Mucho código duplicado
- Difícil cambiar el tiempo de espera global

👉 Solución: **BasePage + reutilización**

BasePage – Patrón de herencia

Idea principal

Todas las páginas heredan de una clase común que contiene:

- WebDriver
- WebDriverWait
- Métodos reutilizables (click, escribir, esperar)

```
public class BasePage {  
  
    protected WebDriver driver;  
    protected WebDriverWait wait;  
  
    public BasePage(WebDriver driver) {  
        this.driver = driver;  
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
    }  
  
    protected WebElement esperarVisible(By locator) {  
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));  
    }  
  
    protected void click(By locator) {  
        esperarVisible(locator).click();  
    }  
  
    protected void escribir(By locator, String texto) {  
        esperarVisible(locator).clear();  
        esperarVisible(locator).sendKeys(texto);  
    }  
  
    protected String obtenerTexto(By locator) {  
        return esperarVisible(locator).getText();  
    }  
}
```

POM avanzado usando BasePage

👉 Beneficios:

- Código más corto
- Menos duplicación
- Esperas centralizadas
- Cambios más simples

```
public class LoginPage extends BasePage {  
  
    private By user = By.id("user-name");  
    private By pass = By.id("password");  
    private By loginBtn = By.id("login-button");  
  
    public LoginPage(WebDriver driver) {  
        super(driver);  
    }  
  
    public void login(String usuario, String password) {  
        escribir(user, usuario);  
        escribir(pass, password);  
        click(loginBtn);  
    }  
}
```


BaseTest – Centralización del WebDriver

👉 Todas las clases de **test** heredan de **BaseTest**.

```
public class BaseTest {  
  
    protected WebDriver driver;  
  
    @BeforeEach  
    void setup() {  
        WebDriverManager.chromedriver().setup();  
        driver = new ChromeDriver();  
        driver.manage().window().maximize();  
    }  
  
    @AfterEach  
    void tearDown() {  
        if (driver != null) {  
            driver.quit();  
        }  
    }  
}
```

Centralización de esperas (Wait Strategy)

Problema

Cada Page define sus propias esperas → inconsistencia.

Solución

- Centralizar esperas en BasePage
- O usar una clase WaitHelper

Ejemplo conceptual:

- Un solo timeout configurable
- Un solo criterio de espera

Utils y Helpers

¿Qué son?

Clases utilitarias para lógica que **no pertenece a una página**.

Ejemplos:

- Generar datos dinámicos
- Capturar screenshots
- Manejo de fechas
- Scroll, JS, acciones especiales

Java Faker (para datos aleatorios)

La librería **Faker** genera datos **falsos** o **aleatorios**. Muy útil para crear datos de prueba realistas.



Funcionalidad:

- Generación de nombres, emails, direcciones, etc.

Repositorio oficial: <https://github.com/DiUS/java-faker>

```
<!-- https://mvnrepository.com/artifact/com.github.javafaker/javafaker -->
<dependency>
  <groupId>com.github.javafaker</groupId>
  <artifactId>javafaker</artifactId>
  <version>1.0.2</version>
</dependency>
```

```
import com.github.javafaker.Faker;

public class FakerHelper {
    public static String generarEmail() {
        Faker faker = new Faker();
        return faker.internet().emailAddress();
    }

    public static String generarNombre() {
        Faker faker = new Faker();
        return faker.name().fullName();
    }
}
```

Java Time (Manejo de fechas)

Java tiene su propio paquete para el manejo de fechas desde **Java 8**: `java.time`.
Es mucho más flexible y poderoso que las clases tradicionales (`Date`, `Calendar`).



Funcionalidad:

- `LocalDate`, `LocalDateTime`, `Duration`, `Period` para operaciones de fechas.

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class DateHelper {
    public static String obtenerFechaHoy() {
        LocalDate hoy = LocalDate.now();
        return hoy.format(DateTimeFormatter.ofPattern("dd/MM/yyyy"));
    }

    public static String agregarDias(int dias) {
        LocalDate fechaFutura = LocalDate.now().plusDays(dias);
        return fechaFutura.format(DateTimeFormatter.ofPattern("dd/MM/yyyy"));
    }
}
```

Scroll y desplazamiento avanzado

El scroll en Selenium a veces no funciona correctamente si solo usas **Actions**. Aquí un ejemplo de **scroll más avanzado** usando **JavaScript**.

Funcionalidad:

- Realizar **scrolls más complejos** o movimientos que no logran con las acciones estándar.

```
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;

public class ScrollHelper {

    public static void scrollToElement(WebDriver driver, By locator) {
        JavascriptExecutor js = (JavascriptExecutor) driver;
        WebElement element = driver.findElement(locator);
        js.executeScript("arguments[0].scrollIntoView(true);", element);
    }
}
```

```
public class DataHelper {  
  
    public static String emailRandom() {  
        return "user" + System.currentTimeMillis() + "@mail.com";  
    }  
}
```

```
public class ScreenshotHelper {  
  
    public static void capturar(WebDriver driver, String nombre) {  
        File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);  
        // copiar archivo (ejemplo conceptual)  
    }  
}
```

Arquitectura recomendada del framework

```
src/test/java
├── base
│   ├── BaseTest.java
│   └── BasePage.java
├── pages
│   ├── LoginPage.java
│   ├── HomePage.java
│   └── CheckoutPage.java
├── tests
│   ├── LoginTest.java
│   └── CompraTest.java
├── utils
│   ├── DataHelper.java
│   └── ScreenshotHelper.java
```


Práctica

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

Ejercicio guiado

Tomar un caso automatizado, refactorizarlo con todo lo aprendido hasta el momento.

Cierre

**COLEGIO
UNIVERSITARIO**



CCC

Centro de Capacitación
Continua

Buenas prácticas

- ❌ No poner asserts en Page Objects
- ❌ No usar `Thread.sleep()`
- ❌ No duplicar locators
- ✅ Pages solo ejecutan acciones
- ✅ Tests solo verifican
- ✅ Un test = un escenario
- ✅ Código legible antes que “ingenioso”

¿Preguntas?

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

¡Gracias!

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua